SMART RECIPE SUGGESTION SYSTEM

Module-wise Technical Documentation

Capstone Project Report

Submitted in partial fulfillment of the requirements for the degree of

Bachelor of Computer Applications (B.C.A.)

Academic Year 2024 - 2025

TABLE OF CONTENTS

Project Overview

This document provides the complete module-wise technical and functional documentation for the Smart Recipe Suggestion System, developed as a capstone project for the Bachelor of Computer Applications programme. The system is designed to assist users in discovering recipes they can prepare using the ingredients they already have at home, thereby reducing food waste and simplifying meal planning.

The core idea of the project is both practical and technically rich. A user enters a list of ingredients available to them, and the system intelligently matches those ingredients against a recipe database, identifies what is missing, suggests possible substitutes, and finally presents the top matching recipes with complete cooking instructions.

The system is divided into five functional modules that work together in a sequential pipeline:

1. User Input Module — Collects and validates the list of available ingredients from the user.
2. Recipe Matching Module — Compares user ingredients against the recipe database and ranks suitable recipes.
3. Missing Ingredient Detection Module — Identifies which required ingredients the user does not have.
4. Alternative Ingredient Suggestion Module — Recommends practical substitutes for missing ingredients.
5. Recipe Display Module — Presents the final recipe details including steps, ingredients, and cooking time.

The system demonstrates core concepts in database design, backend logic, user interface development, and algorithmic thinking, making it well-suited for academic evaluation and viva examination.

Module 1: User Input Module

1.1 Overview

The User Input Module is the starting point of the entire system. Its sole responsibility is to provide the user with a clean and intuitive interface through which they can enter the ingredients they currently have available. This module also performs initial validation on the entered data before forwarding it to the Recipe Matching Module.

A well-designed input module is critical to the overall user experience. If the input process is confusing or error-prone, the rest of the system cannot function correctly regardless of how sophisticated the backend logic is. This module is therefore designed with simplicity and reliability as its primary goals.

1.2 Objectives

- Provide a user-friendly interface for entering one or more ingredient names.
- Accept both typed input and selection from a pre-defined ingredient list.
- Validate each entered ingredient to ensure it is recognisable and correctly spelled.
- Allow the user to add, remove, and review ingredients before submitting.
- Transmit the final validated ingredient list to the server for processing.

1.3 Functional Requirements

- **Ingredient Entry:** The user types ingredient names into an input field. An autocomplete dropdown assists with spelling and recognition.
- **Tag-Based Display:** Each confirmed ingredient appears as a removable tag or chip in the interface, giving the user a clear visual list of what they have entered so far.
- **Input Validation:** The system checks each entry against a master ingredients list. Unrecognised entries are flagged with a suggestion to correct or remove them.
- **Duplicate Prevention:** If the user attempts to add an ingredient that is already in the list, the system silently ignores the duplicate entry.
- **Minimum Requirement:** The system requires at least two ingredients before the search can be initiated, preventing meaningless single-ingredient searches.
- **Submit Action:** A clearly labelled button triggers the submission. The ingredient list is sent as a structured request to the backend server.

1.4 Technical Design

Technology Stack

- Frontend: HTML5, CSS3, JavaScript with autocomplete functionality
- Backend Endpoint: Python Flask route receiving a POST request with a JSON body
- Data Format: JSON array of ingredient name strings
- Database Reference: Ingredients master table used for autocomplete and validation

Input Workflow

6. User opens the application and lands on the main input screen.
7. User types an ingredient name. The autocomplete dropdown shows matching suggestions from the database.
8. User selects or confirms an ingredient. It appears as a tag in the selected ingredients panel.
9. User repeats the process until all available ingredients have been entered.
10. User clicks the Find Recipes button. The validated ingredient list is sent to the server as a POST request.

1.5 Input and Output

- **Sample Input:** Onion, Tomato, Egg, Rice
- **Output:** A validated JSON array forwarded to the Recipe Matching Module: ["Onion", "Tomato", "Egg", "Rice"]

1.6 Output to Next Module

The validated and structured ingredient list is passed as the primary input to the Recipe Matching Module. The list is also stored temporarily in the user session so that subsequent modules can reference the original entries without requiring another network request.

Module 2: Recipe Matching Module

2.1 Overview

The Recipe Matching Module is the intelligence layer of the system. It receives the validated list of available ingredients from the User Input Module and compares them against every recipe stored in the database. The goal is to identify which recipes can be prepared, either fully or partially, using the provided ingredients, and to rank those recipes by suitability.

This module determines the overall usefulness of the system. A naive approach of requiring an exact match would return too few results. A well-designed matching algorithm accounts for partial availability and ranks results in a way that genuinely helps the user decide what to cook.

2.2 Objectives

- Query the recipe database using the user-provided ingredient list.
- Calculate a matching score for each recipe based on the proportion of available ingredients.
- Rank recipes in descending order of matching score.
- Return the top three to four recipes as candidates for further processing.
- Handle cases where no recipes meet a minimum matching threshold gracefully.

2.3 Matching Algorithm

For each recipe in the database, the matching score is calculated using the following logic:

11. Retrieve the complete list of required ingredients for the recipe.
12. Count how many of those required ingredients are present in the user's available list.
13. Divide the matched count by the total required ingredient count to obtain a percentage score.
14. Recipes with a matching score of fifty percent or higher are considered candidates.
15. Candidates are sorted in descending order of their score.
16. The top three to four candidates are selected for output.

For example, if a recipe requires six ingredients and the user has four of them, the matching score is 66.7 percent. If another recipe requires four ingredients and the user has all four, that recipe scores 100 percent and is ranked higher.

2.4 Database Design

Recipes Table

- recipe_id: Unique integer identifier for each recipe.
- recipe_name: Display name of the recipe.
- cuisine_type: Category such as Indian, Chinese, or Continental.
- cooking_time_minutes: Estimated total preparation and cooking time.
- difficulty_level: Enumerated value of Easy, Medium, or Hard.

Recipe Ingredients Table

- recipe_id: Foreign key referencing the Recipes table.
- ingredient_id: Foreign key referencing the Ingredients master table.
- quantity: Amount required, stored as a text string such as two tablespoons.
- is_optional: Boolean flag indicating whether the ingredient is optional or essential.

2.5 Example

- **User Input:** Onion, Tomato, Egg

Matching results returned:

17. Egg Curry — 3 out of 4 required ingredients matched — score 75%
18. Omelette — 2 out of 2 required ingredients matched — score 100%
19. Egg Fried Rice — 3 out of 6 required ingredients matched — score 50%
20. Egg Sandwich — 2 out of 4 required ingredients matched — score 50%

2.6 Output to Next Module

The top three to four ranked recipe objects, including their recipe IDs, names, matching scores, and full ingredient requirements, are passed to the Missing Ingredient Detection Module for gap analysis.

Module 3: Missing Ingredient Detection Module

3.1 Overview

The Missing Ingredient Detection Module examines each of the top-ranked recipes identified by the Recipe Matching Module and determines exactly which ingredients are required but not currently available to the user. This information is essential for two reasons: it helps the user understand what they would need to buy if they choose that recipe, and it feeds directly into the Alternative Ingredient Suggestion Module.

Rather than simply telling the user that a recipe is a partial match, this module provides actionable detail by naming the specific missing items. This transforms a vague match percentage into concrete, useful information.

3.2 Objectives

- For each top-ranked recipe, retrieve the complete list of required ingredients from the database.
- Compare the required list against the user's available ingredient list.
- Produce a clearly structured list of missing ingredients for each recipe.
- Differentiate between essential missing ingredients and optional missing ingredients.
- Provide the missing ingredient data to the Alternative Suggestion Module.

3.3 Detection Logic

21. Receive the list of top-ranked recipes and the user's available ingredients.
22. For each recipe, fetch the full ingredient requirement list from the Recipe Ingredients table.
23. Perform a set difference operation: required ingredients minus available ingredients.
24. The result is the missing ingredients list for that recipe.
25. Flag each missing ingredient as either essential or optional based on the is_optional field.
26. Store the missing ingredients alongside their recipe in a structured result object.

3.4 Example

- **Recipe:** Egg Fried Rice
- **Required Ingredients:** Egg, Onion, Rice, Carrot, Soy Sauce, Spring Onion
- **Available Ingredients:** Egg, Onion, Rice

Missing ingredients identified:

- Carrot — Essential

- Soy Sauce — Essential
- Spring Onion — Optional

3.5 Handling Edge Cases

- **All Ingredients Available:** If a recipe has no missing ingredients, the system marks it as Fully Preparable and skips the alternative suggestion step for that recipe.
- **All Ingredients Missing:** If fewer than the minimum threshold of required ingredients are available, the recipe may be demoted in ranking or removed from results entirely depending on the configured threshold.
- **Optional Ingredients:** Missing optional ingredients are displayed separately in the output with a note that the dish can still be prepared without them.

3.6 Output to Next Module

The structured output of this module is a list of recipe objects, each containing the recipe name, its matching score, and a categorised list of missing ingredients. This is passed to the Alternative Ingredient Suggestion Module, which will attempt to provide substitutes for each missing essential ingredient.

Module 4: Alternative Ingredient Suggestion Module

4.1 Overview

The Alternative Ingredient Suggestion Module addresses the problem of missing ingredients by recommending practical, commonly available substitutes. When a user is missing an ingredient that a recipe requires, this module searches a curated substitutions database and presents one or more alternatives that can be used in its place without significantly altering the outcome of the dish.

This module is what elevates the system from a simple recipe search tool to a genuinely intelligent cooking assistant. By offering alternatives, the system empowers users to cook a wider range of dishes even when their pantry is not fully stocked.

4.2 Objectives

- For each missing essential ingredient identified in Module 3, search the substitutions database for known alternatives.
- Return one to three substitute suggestions ranked by suitability and availability.
- Provide a brief note explaining the substitution and any adjustments the user may need to make.
- Handle cases where no substitute exists gracefully.
- Display the suggestions alongside the missing ingredient in the recipe output.

4.3 Substitutions Database Design

Substitutions Table

- sub_id: Unique identifier for each substitution record.
- original_ingredient_id: Foreign key referencing the ingredient being replaced.
- substitute_ingredient_id: Foreign key referencing the suggested substitute.
- suitability_score: A numeric score from 1 to 10 indicating how well the substitute works.
- adjustment_note: A short text explanation of how to use the substitute differently, if needed.

4.4 Suggestion Logic

27. Receive the list of missing essential ingredients from Module 3.
28. For each missing ingredient, query the Substitutions table using the original_ingredient_id.
29. Retrieve all available substitutes, ordered by suitability_score in descending order.
30. Return the top three substitutes for each missing ingredient.
31. If no substitutes are found, mark the ingredient as No Alternative Available.

4.5 Example Substitutions

- **Missing: Soy Sauce:** Alternatives: Coconut Aminos (score 9), Worcestershire Sauce (score 7), Salt and Vinegar mixture (score 5).
- **Missing: Carrot:** Alternatives: Capsicum (score 8), Zucchini (score 7), Beetroot (score 5).
- **Missing: Butter:** Alternatives: Refined Oil (score 9), Ghee (score 9), Coconut Oil (score 7).

4.6 Contextual Filtering

The substitution logic also considers the cuisine type of the recipe. For example, if the recipe is an Indian dish, the system prioritises substitutes that are commonly found in Indian households. This contextual filtering improves the practical relevance of the suggestions and avoids recommending exotic substitutes that may be equally unavailable.

4.7 Output to Next Module

The complete result object, now containing the recipe details, missing ingredients, and their respective substitution suggestions, is passed to the Recipe Display Module for final presentation to the user.

Module 5: Recipe Display Module

5.1 Overview

The Recipe Display Module is the final stage of the pipeline and the primary interface through which the user consumes the output of the entire system. It receives the enriched recipe data produced by the preceding four modules and presents it in a clear, structured, and visually accessible format. The user can browse the top matching recipes, review what they have and what is missing, see substitution suggestions, and follow step-by-step cooking instructions.

This module is purely presentational in nature. It does not perform any computation or data retrieval of its own. Its quality is judged entirely on how clearly and usefully it communicates complex information to the user.

5.2 Objectives

- Display three to four recipe cards summarising the top matching results.
- For each selected recipe, show the complete recipe detail view.
- Present required ingredients, clearly distinguishing available and missing items.
- Show alternative ingredient suggestions alongside each missing item.
- Display numbered preparation steps in an easy-to-follow format.
- Show total cooking time and difficulty level.

5.3 Display Components

Recipe Card List

The results screen shows three to four recipe cards. Each card displays the recipe name, cuisine type, matching score as a percentage, estimated cooking time, and a brief description. The user taps or clicks a card to open the full recipe detail view.

Ingredients Section

The full recipe view displays two clearly separated lists. The first list, labelled Available Ingredients, shows all required ingredients that the user already has, displayed in green or with a checkmark. The second list, labelled Missing Ingredients, shows all required items that the user does not have, along with the suggested substitutes directly beneath each missing item. Optional missing ingredients are shown in a separate sub-section labelled Optional.

Preparation Steps Section

The cooking instructions are displayed as a numbered list. Each step is written in plain, simple language. Steps that involve timing, such as fry for three minutes, include a visual indicator of the duration. The user can tap a step to mark it as completed, helping them track their progress while cooking.

Recipe Summary Panel

At the top of the detail view, a summary panel shows the recipe name, a representative image if available, the total preparation time, the cooking time, the total time, the difficulty level, and the number of servings the recipe produces.

5.4 Output Example

Recipe Name: Egg Fried Rice

Required Ingredients:

- Egg — Available
- Onion — Available
- Rice — Available
- Carrot — Missing | Alternative: Capsicum or Zucchini
- Soy Sauce — Missing | Alternative: Coconut Aminos or Worcestershire Sauce
- Spring Onion — Optional, Missing | Alternative: Regular Onion

Preparation Steps:

32. Cook rice and allow it to cool completely. Freshly cooked warm rice tends to clump.
33. Beat the eggs and scramble them in a lightly oiled pan over medium heat. Set aside.
34. In the same pan, add oil and saute the chopped onion and carrot until softened.
35. Add the cooled rice to the pan and stir well to combine with the vegetables.
36. Add soy sauce and mix thoroughly until the rice is evenly coated.
37. Add the scrambled eggs back into the pan and fold gently into the rice.
38. Garnish with spring onion if available. Serve hot.

- **Preparation Time:** 10 minutes
- **Cooking Time:** 15 minutes
- **Total Time:** 25 minutes
- **Difficulty:** Easy
- **Serves:** 2 persons

5.5 Technical Design

- Frontend rendered with HTML5 and CSS3. Recipe cards use a responsive grid layout.
- Dynamic content injected using JavaScript fetch API calls to the Flask backend.
- Step completion tracking managed in browser local storage during the cooking session.
- Recipe images stored in a server-side media folder and referenced by file path in the database.

5.6 Accessibility Considerations

The display module is designed to be usable in a kitchen environment, where users may have wet hands or limited attention. Text is sized generously for readability at a distance. The step-by-step format minimises cognitive load. The ingredient availability colour coding provides instant visual clarity without requiring the user to read each line carefully.

Conclusion

This document has provided the complete module-wise functional and technical documentation for the Smart Recipe Suggestion System, a B.C.A. capstone project that demonstrates the practical application of database management, backend development, algorithmic thinking, and user interface design within a single integrated system.

The five modules — User Input, Recipe Matching, Missing Ingredient Detection, Alternative Ingredient Suggestion, and Recipe Display — each address a distinct and meaningful sub-problem. Together, they form a pipeline that takes raw user input and transforms it into personalised, actionable cooking guidance. The system is not merely an academic exercise but a genuinely useful application that solves a real everyday problem.

From a technical standpoint, the project covers key concepts including relational database design with multiple related tables, set-based algorithmic operations for ingredient matching and gap detection, RESTful API design with Python Flask, dynamic frontend rendering with JavaScript, and user experience considerations for practical usability.

Possible future enhancements include integration with nutrition data APIs to display calorie information for each recipe, a user profile system that remembers frequently used ingredients and dietary preferences, and a machine learning layer that improves recipe ranking based on past user choices and ratings.